a. The two genAi used are Claude and Copilot

b. Both were accessed on the web

c. Write a recursive, object-oriented C++ program that solves Sudoku puzzles using depth-first search with backtracking. Use classes with member functions rather than standalone functions (main() should only create an object and run it).

Sudoku rules: fill every cell with a digit 1-9 so that each digit appears exactly once in each row, each column, and each 3x3 block.

Approach (brute force, not logical deduction): find an empty cell and compute its candidate digits from its row, column, and block. If there are no candidates, return "no solution found" and backtrack. Otherwise try the first candidate and recurse; if that fails, try the next, and so on. When all 81 cells are filled, a solution has been found.

Input: five files, puzzle1.txt through puzzle5.txt. Each holds a 9x9 grid with space-separated digits, and an underscore (_) marks a blank square. Example:

5 _ _ _ _ _ 1 7 _

1 _ 6 5 _ 9 _ 4 _

4 7 2 1 _ 6 _ _ _

9 _ _ _ _ _ 5 _ _

_ 1 8 _ 9 5 4 _ _

6 _ _ 4 _ 2 3 8 9

_ 4 _ _ _ _ 9 3 _

_ 9 _ 7 _ 3 _ 5 _

2 6 3 9 5 8 7 1 4

Output, for each puzzle file:

- The puzzle file name (e.g., puzzle1.txt)

- The puzzle as printed from the file

- The solution, or "No solution found" if none exists

A puzzle may have more than one solution. If so, print them all.

d. Claudes code:

```
// Sudoku solver: recursive depth-first search with backtracking (all solutions).
// Build: g++ -o claude claude.cpp
// Usage: ./claude puzzle1.txt puzzle2.txt puzzle3.txt puzzle4.txt puzzle5.txt
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// A 9x9 board. 0 represents a blank ('_' in the files).
class Board {
public:
using Cells = std::array<std::array<int, 9>, 9>;

Board() { cells_.fill({}); }

// Returns false if the file can't be opened or isn't a 9x9 puzzle.
bool load(const std::string& path) {
std::ifstream in(path);
if (!in) return false;
std::string tok;
int count = 0;
while (in >> tok && count < 81) {
cells_[count / 9][count % 9] = (tok == "_") ? 0 : std::stoi(tok);
++count;
}
return count == 81;
}

int get(int r, int c) const { return cells_[r][c]; }
void set(int r, int c, int v) { cells_[r][c] = v; }

// Digits that may legally go in cell (r, c), given the rest of the board.
std::vector<int> candidates(int r, int c) const {
```

```
bool used[10] = {false};
for (int i = 0; i < 9; ++i) {
used[cells_[r][i]] = true;
used[cells_[i][c]] = true;
}
int br = 3 * (r / 3), bc = 3 * (c / 3);
for (int i = br; i < br + 3; ++i)
for (int j = bc; j < bc + 3; ++j)
used[cells_[i][j]] = true;
std::vector<int> out;
for (int d = 1; d <= 9; ++d)
if (!used[d]) out.push_back(d);
return out;
}

// True if the given clues don't already conflict with each other.
bool isConsistent() {
for (int r = 0; r < 9; ++r)
for (int c = 0; c < 9; ++c) {
int v = cells_[r][c];
if (v == 0) continue;
cells_[r][c] = 0;
bool ok = false;
for (int d : candidates(r, c)) if (d == v) ok = true;
cells_[r][c] = v;
if (!ok) return false;
}
return true;
}

// Finds the empty cell with the fewest candidates. Returns false if full.
bool findEmpty(int& row, int& col, std::vector<int>& cands) const {
bool found = false;
for (int r = 0; r < 9; ++r)
for (int c = 0; c < 9; ++c) {
if (cells_[r][c] != 0) continue;
std::vector<int> cs = candidates(r, c);
if (!found || cs.size() < cands.size()) {
found = true;
row = r; col = c; cands = cs;
if (cands.size() <= 1) return true;
}
}
```

```
return found;
}

void print(std::ostream& os) const {
for (int r = 0; r < 9; ++r) {
for (int c = 0; c < 9; ++c) {
if (c) os << ' ';
if (cells_[r][c]) os << cells_[r][c]; else os << '_';
}
os << '\n';
}
}

private:
Cells cells_;
};

// Depth-first search with backtracking; collects every solution.
class Solver {
public:
explicit Solver(const Board& puzzle) : board_(puzzle) {}

const std::vector<Board>& solveAll() {
solutions_.clear();
if (board_.isConsistent()) search();
return solutions_;
}

private:
void search() {
int r = 0, c = 0;
std::vector<int> cands;
if (!board_.findEmpty(r, c, cands)) { // k == 81: solved
solutions_.push_back(board_);
return;
}
for (int d : cands) { // empty list -> dead end
board_.set(r, c, d);
search(); // recurse with k+1 cells filled
board_.set(r, c, 0); // backtrack
}
}
```

```
Board board_;
std::vector<Board> solutions_;
};

// Runs the solver over a list of puzzle files and prints the results.
class Application {
public:
Application(int argc, char** argv) {
for (int i = 1; i < argc; ++i) files_.push_back(argv[i]);
if (files_.empty())
for (int i = 1; i <= 5; ++i)
files_.push_back("puzzle" + std::to_string(i) + ".txt");
}

int run() {
for (const std::string& name : files_) {
std::cout << name << '\n';
Board puzzle;
if (!puzzle.load(name)) {
std::cout << "Could not read puzzle file\n\n";
continue;
}
puzzle.print(std::cout);
std::cout << '\n';

Solver solver(puzzle);
const std::vector<Board>& sols = solver.solveAll();
if (sols.empty()) std::cout << "No solution found\n";
for (size_t i = 0; i < sols.size(); ++i) {
if (sols.size() > 1)
std::cout << "Solution " << i + 1 << " of " << sols.size() << ":\n";
sols[i].print(std::cout);
std::cout << '\n';
}
std::cout << "--------------------\n";
}
return 0;
}

private:
std::vector<std::string> files_;
};
```

```
int main(int argc, char** argv) {
return Application(argc, argv).run();
}
```

e. The program is correct for all sudoku inputs that fit the requirements. The program has a worst-case time complexity of O(9^n) where n is blank cells, and I could not find a way to improve it. The space complexity is O(1). I also could not find a way to improve it. As for maintainability, all the classes have reasonable names, but comments are not up to standard.

d(2): Copilots code:

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class SudokuPuzzle
{
private:
char board[9][9];
int solutionCount;

public:
SudokuPuzzle()
{
solutionCount = 0;
}

bool loadPuzzle(string fileName)
{
ifstream fin(fileName);

if (!fin)
return false;

for (int row = 0; row < 9; row++)
{
for (int col = 0; col < 9; col++)
{
fin >> board[row][col];
}
```

```
}

fin.close();
return true;
}

void printPuzzle()
{
for (int row = 0; row < 9; row++)
{
for (int col = 0; col < 9; col++)
{
cout << board[row][col] << " ";
}

cout << endl;
}
}

bool validMove(int row, int col, char value)
{
// Check row
for (int c = 0; c < 9; c++)
{
if (board[row][c] == value)
return false;
}

// Check column
for (int r = 0; r < 9; r++)
{
if (board[r][col] == value)
return false;
}

// Check 3x3 block
int startRow = (row / 3) * 3;
int startCol = (col / 3) * 3;

for (int r = startRow; r < startRow + 3; r++)
{
for (int c = startCol; c < startCol + 3; c++)
```

```
{
if (board[r][c] == value)
return false;
}
}

return true;
}

bool findBlank(int &row, int &col)
{
for (row = 0; row < 9; row++)
{
for (col = 0; col < 9; col++)
{
if (board[row][col] == '_')
return true;
}
}

return false;
}

void solve()
{
int row;
int col;

if (!findBlank(row, col))
{
solutionCount++;

cout << "\nSolution " << solutionCount << endl;
printPuzzle();
cout << endl;

return;
}

for (char digit = '1'; digit <= '9'; digit++)
{
if (validMove(row, col, digit))
```

```
{
board[row][col] = digit;

solve(); // Recursive call

board[row][col] = '_'; // Backtracking
}
}
}

void startSolving()
{
solutionCount = 0;

solve();

if (solutionCount == 0)
{
cout << "\nNo solution found.\n";
}
}
};

class PuzzleDriver
{
public:
void run()
{
string files[5] =
{
"puzzle1.txt",
"puzzle2.txt",
"puzzle3.txt",
"puzzle4.txt",
"puzzle5.txt"
};

for (int i = 0; i < 5; i++)
{
SudokuPuzzle sudoku;

cout << "\n==================================\n";
```

```
cout << "File: " << files[i] << endl;
cout << "================================\n";

if (!sudoku.loadPuzzle(files[i]))
{
cout << "Unable to open file.\n";
continue;
}

cout << "\nOriginal Puzzle:\n";
sudoku.printPuzzle();

cout << "\nSolutions:\n";
sudoku.startSolving();
}
}
};

int main()
{
PuzzleDriver program;
program.run();

return 0;
}
```

e. Copilots code had no problems as far as I could find. The worst-case time complexity is O(9^n) where n is the number of blanks. Space complexity is O(1) and isn't a factor. The code has clear function names but not many comments.

f. I chose Claudes code because it was less complex and still completed the task perfectly. I found it easier to understand and it had less lines.

g. I tried to improve the code, but I couldn't find any concrete improvements that wouldn't remove functionality. Instead I went through the code and added comments.